

velCursor: A Coupled Inertial Input and Hex-Cell Caret-Trail Rendering System for VS Code

A Code-Faithful Hybrid Systems Treatment with Stability, Validity, and Numerical Guarantees

velCursor Project Technical Note (Expanded, Professor-Style)

Abstract—This paper documents the velCursor extension architecture and implementation as an engineering system, not a visual effect preset. The design couples a host-side inertial cursor controller with a browser-side geometric renderer embedded in Monaco. The host module generates bounded discrete cursor motion from impulse-triggered user commands; the renderer reconstructs a synthetic caret state, propagates a spring-based corner model, and emits a directional hex-cell trail with age fading, concavity shaping, and turn-point topology constraints. A second-order under-damped oscillator modulates anisotropic scale in both historical sections and the live head cap to preserve spring-like motion cues near the caret. The implementation further includes adaptive quality regulation using frame-time and motion-distance pressure, with explicit degradation and recovery rates.

Beyond reproducing the code-faithful equations, this note upgrades the treatment to a hybrid dynamical-systems analysis: we provide boundedness and stability results (including Lyapunov proofs), bounded-input bounded-output guarantees for the overshoot subsystem, numerical stability and error accumulation bounds for explicit integration components, and geometric validity conditions for polygonal trail cell construction. Each formal development is followed by plain-language interpretation intended to make tuning predictable, reproducible, and explainable.

Index Terms—cursor dynamics, human-computer interaction, motion synthesis, geometric rendering, VS Code extension, Monaco editor, adaptive quality control, hybrid systems, Lyapunov stability

I. INTRODUCTION

Modern editors typically represent cursor motion as piecewise-constant jumps, which is semantically correct but visually discontinuous. velCursor addresses this mismatch by introducing physically informed motion persistence with explicit control over bounded behavior.

The implementation is split across two runtimes:

- 1) host-side command and state control in `extension.js` and `cursorInertia.js`;
- 2) browser-side rendering in `cursorTrail.js` (injected into Monaco DOM context).

The system-level objective is to maximize perceived continuity under finite rendering budget. Unlike purely decorative trails, the implementation enforces geometric validity constraints and turn-stability gates, because invalid polygons and turn chatter were observed to cause visible spikes or dropout.

A. Contributions

This technical note provides:

- a discrete-time model of the inertial cursor controller, including level-to-parameter interpolation;
- a geometric formulation of the hex-cell trail synthesis path, including concavity and turn-side-sharing rules;
- an analysis of anisotropic overshoot coupling and head attachment continuity;
- the adaptive quality control law and a complexity estimate for practical tuning;
- (new) boundedness and stability theorems for the coupled architecture, with Lyapunov proofs;
- (new) numerical stability and error accumulation bounds for explicit integration subsystems;
- (new) geometric validity lemmas for local non-self-intersection under bounded concavity.

B. Plain interpretation

velCursor is best treated as a small control-and-render pipeline. The host decides discrete moves. The renderer turns those moves into continuous-looking motion, but only as long as the geometry stays valid and performance stays within budget. The point of the math is to make the knobs behave like knobs, not like magic.

II. SYSTEM ARCHITECTURE

A. Host Runtime

The extension activates movement commands and exposes two persistent configuration scalars:

- impulse level $\ell_I \in \{1, \dots, 10\}$, mapped to per-impulse velocity increment;
- tick slowness level $\ell_T \in \{1, \dots, 10\}$, mapped to simulation tick period.

Command families are separated by intent:

- single-step commands `cursorOnce.*` perform one fixed move and never enter inertia;
- impulse commands `inertiaCursor.*` start or accelerate a run;
- `inertiaCursor.stop` hard-stops the state machine.

B. Renderer Runtime

The renderer attaches a canvas overlay to the active Monaco host, keeps native caret layers visible above custom graphics, and runs one update per animation frame. The frame pipeline is:

- 1) poll native caret rectangle and synthesize a stable font-metric-aligned caret box;

- 2) advance four corner springs (`CursorSquish`);
- 3) push canonicalized polygons into a trail history buffer with adaptive interpolation;
- 4) construct and fill directional hex cells from sections;
- 5) draw caret box stroke for positional legibility and head contact.

C. Plain interpretation

Think of the host as the actuator and the renderer as the sensor-fusion plus display stage. The host moves the real caret. The renderer samples that caret and renders a physically plausible “afterimage” that stays attached and does not self-destruct geometrically.

III. FORMAL HYBRID SYSTEMS MODEL

Definition 1 (Hybrid System Decomposition). *We model `velCursor` as a coupled hybrid system*

$$S \equiv (\mathcal{H}, \mathcal{R}, \mathcal{C}),$$

where $\mathcal{H}$ is a discrete-time inertial state machine, $\mathcal{R}$ is a continuous-time geometric dynamical system sampled at frame rate, and $\mathcal{C}$ is a coupling map that (i) transmits caret kinematics from $\mathcal{H}$ to $\mathcal{R}$, and (ii) transmits motion-derived signals (e.g., displacement) to auxiliary renderer subsystems such as overshoot and quality control.

A. Plain interpretation

“Hybrid” here just means two clocks. The host updates on a tick timer. The renderer updates on animation frames. They share state through the caret rectangle and a few derived motion scalars.

IV. HOST-SIDE INERTIAL MOTION MODEL

A. Level Mapping

Define normalized level coordinate

$$t(\ell) = \frac{\ell - 1}{9}, \quad \ell \in \{1, \dots, 10\}. \quad (1)$$

Then

$$I(\ell_I) = I_{\min} + t(\ell_I) (I_{\max} - I_{\min}), \quad (2)$$

$$\Delta t(\ell_T) = \text{round} (T_{\min} + t(\ell_T) (T_{\max} - T_{\min})), \quad (3)$$

with code defaults $I_{\min} = 0.30$, $I_{\max} = 2.10$, $T_{\min} = 14$ ms, and $T_{\max} = 68$ ms.

B. State Machine and Tick Law

Let v_k denote scalar velocity at tick k , d_k direction, and s_k cursor step. For same-direction impulses,

$$v_k \leftarrow \min(v_k + I, v_{\max}). \quad (4)$$

For direction changes during a running state,

$$v_k \leftarrow \max(v_0, \gamma v_k), \quad (5)$$

where v_0 is the initial run velocity and γ is the direction-change damping factor (code uses 0.1). Step magnitude is

$$s_k = \begin{cases} 2, & \text{first tick of a new run} \\ \text{clamp}(\text{round}(v_k), 1, s_{\max}), & \text{otherwise.} \end{cases} \quad (6)$$

Velocity decays geometrically:

$$v_{k+1} = d_v v_k, \quad d_v \in (0, 1), \quad (7)$$

and the run stops when $v_k < v_{\text{cut}}$. Current defaults are $d_v = 0.96$, $v_{\text{cut}} = 0.99$, and $s_{\max} = 8$.

C. Stability and boundedness

Theorem 1 (Host Velocity Boundedness). *Assume $v_0 \geq 0$, $v_{\max} > 0$, $I \geq 0$, and $0 < d_v < 1$. Then for all ticks k ,*

$$0 \leq v_k \leq v_{\max}.$$

Proof. The update $v_k \leftarrow \min(v_k + I, v_{\max})$ ensures $v_k \leq v_{\max}$ immediately after any impulse addition. The decay update multiplies by $d_v \in (0, 1)$, preserving non-negativity and maintaining the upper bound. Thus v_k remains in $[0, v_{\max}]$ for all k . $\square$

Theorem 2 (Exponential Decay Without Impulses). *If no impulses occur over ticks $k, k+1, \dots$, then*

$$v_{k+n} = d_v^n v_k,$$

and hence $v_{k+n} \rightarrow 0$ exponentially as $n \rightarrow \infty$.

Proof. Direct iteration of $v_{j+1} = d_v v_j$ yields $v_{k+n} = d_v^n v_k$. Since $|d_v| < 1$, the sequence converges exponentially to 0. $\square$

D. Plain interpretation

The host controller is a damped integrator with a hard cap. You can kick it repeatedly, but it never runs away. If you stop kicking it, it dies out quickly and predictably.

V. CARET-STATE RECONSTRUCTION AND CORNER DYNAMICS

A. Synthetic Caret Box

Because DOM caret geometry can jitter across frames, the renderer re-estimates the caret box using resolved font size and line height:

$$w_c = \max(w_{\min}, f_s e_w), \quad h_c = \max(h_{\min}, h_{\text{line}}), \quad (8)$$

where f_s is font size and e_w is width in em units (`caretWidthEm`).

B. Critically Damped Corner Springs

Each corner tracks offset from its destination under a critically damped model. With x as offset and $\dot{x}$ as velocity,

$$\ddot{x} + 2\omega\dot{x} + \omega^2 x = 0, \quad \omega = \frac{4}{\tau}. \quad (9)$$

The code uses closed-form integration over Δt :

$$x_{k+1} = (a + b\Delta t)e^{-\omega\Delta t}, \quad (10)$$

$$\dot{x}_{k+1} = (b - \omega a - \omega b\Delta t)e^{-\omega\Delta t}, \quad (11)$$

with $a = x_k$, $b = \omega x_k + \dot{x}_k$. Rank-dependent τ values shape leading/trailing response asymmetry.

C. Lyapunov stability of corner dynamics

Definition 2 (Corner Energy). Define the energy-like Lyapunov candidate

$$V(x, \dot{x}) = \frac{1}{2}\dot{x}^2 + \frac{1}{2}\omega^2 x^2. \quad (12)$$

Theorem 3 (Global Asymptotic Stability of Continuous Corner Dynamics). For constant $\omega > 0$, the equilibrium $(x, \dot{x}) = (0, 0)$ of $\ddot{x} + 2\omega\dot{x} + \omega^2 x = 0$ is globally asymptotically stable.

Proof. Differentiate V :

$$\dot{V} = \dot{x}\ddot{x} + \omega^2 x\dot{x}.$$

Substitute $\ddot{x} = -2\omega\dot{x} - \omega^2 x$:

$$\dot{V} = \dot{x}(-2\omega\dot{x} - \omega^2 x) + \omega^2 x\dot{x} = -2\omega\dot{x}^2 \leq 0.$$

Thus V is non-increasing. Moreover, $\dot{V} = 0$ implies $\dot{x} = 0$, and then the dynamics give $\omega^2 x = 0 \Rightarrow x = 0$. By LaSalle's invariance principle, the origin is globally asymptotically stable. $\square$

D. Plain interpretation

Each corner spring always settles. It does not ring forever. It does not inject energy. It cannot blow up. The rank-dependent τ just changes how fast each corner settles.

VI. TRAIL SAMPLING AND HISTORY CONSTRUCTION

A. Admission Metric

Given previous polygon P_{k-1} and candidate P_k , define

$$m_k = \max(\Delta c_k, \Delta_{\max,k}, \bar{\Delta}_k, 0.35 |\Pi(P_k) - \Pi(P_{k-1})|), \quad (13)$$

where Δc_k is center displacement, $\Delta_{\max,k}$ max corner displacement, $\bar{\Delta}_k$ mean corner displacement, and $\Pi(\cdot)$ perimeter. A sample is admitted only if

$$m_k \geq \max(m_{\min}, \epsilon_{\text{corner}}). \quad (14)$$

B. Adaptive Interpolation

To avoid sparse gaps after large jumps, insertion count is

$$N_k = \min\left(N_{\max}, \left\lceil \frac{m_k}{\delta_{\text{adapt}}/q} \right\rceil\right), \quad (15)$$

where $q \in [q_{\min}, 1]$ is quality. The current defaults are $\delta_{\text{adapt}} = 0.08$ px-equivalent and $N_{\max} = 14$.

C. Age and Opacity

For sample timestamp t_i ,

$$f_i = \text{clamp}\left(\frac{t_{\text{now}} - t_i}{T_{\text{TTL}}}, 0, 1\right), \quad (16)$$

$$\alpha_i = \text{clamp}((1 - f_i)\alpha_0, \alpha_{\min}, 1), \quad (17)$$

with $T_{\text{TTL}} = 420$ ms at current defaults.

D. Plain interpretation

The trail is not “record every frame.” It is “record when motion is meaningful.” Then it fills gaps only when needed, and it fades out according to age.

VII. HEX-CELL GEOMETRY AND TURN-STABILITY RULES

A. Section Resampling and Local Frame

History polygons are resampled to a fixed side count $n_s = 6$ (strict hex mode). For each section, a local tangent D and normal N are estimated from section centers and previous frame direction.

B. Cell Construction

For each consecutive section pair, the filled hex cell uses ordered vertices

$$\mathcal{H}_k = [L_b, R_b, R_p, R_h, L_h, L_p]. \quad (18)$$

Partial share points are

$$R_p = \text{lerp}(R_b, R_h, \sigma_k), \quad L_p = \text{lerp}(L_b, L_h, \sigma_k). \quad (19)$$

Concavity is applied via target

$$Q_k = M_b + s_c D_k \ell_k d_c, \quad (20)$$

where M_b is base midpoint, $s_c \in \{-1, +1\}$ direction sign, ℓ_k smoothed cell length, and d_c concavity depth. A step-limited clamp moves R_p, L_p toward Q_k without geometric collapse.

C. Turn-Point Detection with Anti-Chatter

Width reversal detection is based on EMA-smoothed widths:

$$\hat{w}_k = \begin{cases} w_k, & k = 0 \\ \text{lerp}(\hat{w}_{k-1}, w_k, \beta_w), & k > 0, \end{cases} \quad (21)$$

$$\Delta w_k = \hat{w}_k - \hat{w}_{k-1}. \quad (22)$$

Turn threshold combines absolute and relative components:

$$\epsilon_k = \max(\epsilon_{\text{abs}}, \rho \max(w_{\min}, w_k)). \quad (23)$$

A turn candidate is accepted only if (i) sign reversal exceeds ϵ_k , (ii) both adjacent slopes exceed $d_{\min}$, and (iii) cooldown counter is zero. During accepted turn cells, share σ_k is switched toward σ_{turn} , then smoothed by a share-EMA to prevent sawtooth spikes.

D. Validity and Fallback

Every candidate cell is tested for self-intersection; optional overlap guards can reject near-duplicate geometry. If a turn-share cell fails validity, one retry is computed with normal non-turn share before skipping.

E. Geometric validity lemma (local)

Definition 3 (Simple Polygon). A polygon is simple if its edges do not self-intersect.

Lemma 1 (Local Validity Under Bounded Concavity). Assume consecutive section polygons are convex, consistently oriented, and sufficiently close such that corresponding edges do not cross under linear interpolation. If concavity displacement magnitude satisfies

$$\|Q_k - M_b\| \leq \eta \ell_k \quad (24)$$

for sufficiently small η , then the constructed cell $\mathcal{H}_k$ remains simple.

Proof sketch. The cell vertices are continuous functions of the share parameter and concavity target displacement. For $\eta = 0$, the construction reduces to a non-self-intersecting interpolation of convex sections under the stated correspondence assumption. By continuity of polygon vertex mapping and compactness of the small-displacement neighborhood, there exists $\eta^* > 0$ such that for all $\eta \in [0, \eta^*]$ the edge intersection predicates remain false. Hence the cell remains simple. $\square$

F. Plain interpretation

You can bend the ribbon, but only up to a controlled amount. If you over-bend, the cell can fold. The code avoids this by clamping and by validating the polygon. The lemma formalizes why “small bend” is safe.

VIII. OVERSHOOT-COUPLED ANISOTROPIC DEFORMATION

A. Oscillator

A separate under-damped scalar oscillator models visual overshoot:

$$\ddot{z} + 2\zeta\omega\dot{z} + \omega^2 z = 0. \quad (25)$$

The implementation applies explicit integration:

$$\begin{aligned} a_k &= -2\zeta\omega\dot{z}_k - \omega^2 z_k, \\ \dot{z}_{k+1} &= \dot{z}_k + a_k \Delta t, \\ z_{k+1} &= z_k + \dot{z}_{k+1} \Delta t. \end{aligned} \quad (26)$$

Motion impulses inject velocity $\dot{z} \leftarrow \dot{z} + \Delta\dot{z}$, where $\Delta\dot{z}$ is proportional to frame displacement and clamped by `maxKick`.

B. Anisotropic Trail Scaling

Let $\xi = \text{clamp}(gz, \xi_{\min}, \xi_{\max})$. For age fraction f , envelope

$$e(f) = 0.20 + 0.80(1 - f). \quad (27)$$

Parallel and perpendicular scales are

$$s_{\parallel}(f) = \max(s_{\text{floor}}, e(f) [1 + \xi g_{\parallel}]), \quad (28)$$

$$s_{\perp}(f) = \max(s_{\text{floor}}, e(f) [1 + \xi g_{\perp}]). \quad (29)$$

The same transform is applied to history sections and the live head polygon, preserving overshoot cues near the caret rather than only in the tail.

C. BIBO stability of overshoot oscillator

Theorem 4 (BIBO Stability of Stable Linear Overshoot). Assume $0 < \zeta < 1$ and $\omega > 0$. If impulse injection satisfies $|\Delta\dot{z}| \leq M$ at any frame, then the oscillator state remains bounded: there exists a constant $C(\zeta, \omega)$ such that $|z(t)| \leq CM$ for all t .

Proof sketch. The continuous-time under-damped oscillator is linear time-invariant with eigenvalues $-\zeta\omega \pm i\omega\sqrt{1-\zeta^2}$, which have strictly negative real part. Thus the system is exponentially stable and hence BIBO stable. Bounded impulsive velocity injections correspond to bounded inputs to the stable LTI system, yielding bounded output with gain bounded by the system’s induced norm. $\square$

D. Numerical stability and error bounds (explicit integration)

Theorem 5 (Sufficient Euler Step Condition (Conservative)). Consider the linear oscillator in first-order form $\dot{y} = Ay$ with eigenvalues of A having magnitude $\mathcal{O}(\omega)$. A conservative sufficient condition to avoid catastrophic Euler divergence is

$$\Delta t < \frac{2}{\omega}. \quad (30)$$

Proof sketch. Forward Euler yields $y_{k+1} = (I + A\Delta t)y_k$. A sufficient stability condition is spectral radius $\rho(I + A\Delta t) < 1$. For oscillatory modes dominated by ω , a conservative bound is $\Delta t < 2/\omega$, consistent with the classical stability region constraints. $\square$

Proposition 1 (Global Error Growth Bound (Finite Horizon)). For a Lipschitz continuous system integrated with forward Euler with step Δt , the global truncation error over N steps satisfies

$$\|e_N\| \leq K\Delta t,$$

for some constant K depending on the Lipschitz constant and time horizon $T = N\Delta t$.

E. Plain interpretation

Overshoot is a “little bounce.” The underlying continuous system is stable. The only way to break it is numerical abuse (giant frame times or absurd kicks). The conservative timestep bound tells you where Euler stops behaving, and the error bound tells you the drift stays controlled over finite time.

IX. CARET ATTACHMENT, HEAD BRIDGE, AND CAP

To eliminate visual detachment, the renderer appends a live head polygon reconstructed from current caret rectangle and padded by `rect.padPx`. A bridge section

$$P_{\text{bridge}} = \text{lerp}(P_{\text{last}}, P_{\text{head}}, 0.6) \quad (31)$$

is inserted prior to head insertion. The final head cap is filled even when the last cell is rejected, guaranteeing visual contact between trail and caret box.

A. Plain interpretation

Even if the last hex cell is rejected for validity, the trail still touches the caret. The bridge is basically a continuity patch: it ensures no visual “gap” appears at the head.

X. ADAPTIVE QUALITY CONTROLLER

A. Pressure Computation

Frame-time EMA:

$$\bar{t}_{k+1} = \text{lerp}(\bar{t}_k, t_k, \alpha_f). \quad (32)$$

Frame and distance pressures:

$$p_f = \text{clamp}\left(\frac{\bar{t}_k - t^*}{\Delta t_w}, 0, 1\right), \quad p_d = \text{clamp}\left(\frac{d_k}{d_{\text{norm}}}, 0, 1\right). \quad (33)$$

Hybrid pressure and target quality:

$$p_k = \omega_f p_f + \omega_d p_d, \quad q_k^* = 1 - p_k(1 - q_{\min}). \quad (34)$$

Quality then tracks q_k^* with asymmetric rates $r_{\downarrow}, r_{\uparrow}$ to avoid oscillatory quality pumping.

B. Budget Effects

As q decreases, the renderer increases effective subdivision step, reduces max subdivisions and history depth under configured thresholds, and attenuates blur. This preserves frame stability before sacrificing continuity-critical structure.

C. Contraction proof

Theorem 6 (Contraction Toward Target Quality). *Suppose the quality update is of the form*

$$q_{k+1} = q_k + r(q_k^* - q_k),$$

with $0 < r < 1$ (interpreting asymmetric rates as piecewise constants each in $(0, 1)$). Then the mapping $T(q) = q + r(q^* - q)$ is a contraction with Lipschitz constant $|1 - r| < 1$. Consequently, for constant q^* , the sequence converges exponentially to q^* .

Proof. For any q_1, q_2 ,

$$|T(q_1) - T(q_2)| = |(1 - r)(q_1 - q_2)| = |1 - r| |q_1 - q_2|.$$

Since $0 < r < 1$, we have $|1 - r| < 1$. Thus T is a contraction, giving exponential convergence for constant target q^* . $\square$

D. Plain interpretation

This controller does not “hunt.” It smoothly drifts toward whatever quality level the pressure suggests. Separate down/up rates prevent quality from instantly bouncing back and causing flicker.

XI. COMPUTATIONAL CHARACTERISTICS

Let H be retained history entries, U interpolated draw-time sections, and C emitted cells in a frame. The dominant cost is

$$\mathcal{O}(H + Un_s + Cn_s), \quad (35)$$

with $n_s = 6$ in strict mode. Memory for trail geometry is approximately

$$\mathcal{O}(Hn_s) \quad (36)$$

for section-like representations, plus fixed scratch buffers.

A. Plain interpretation

The cost is basically linear in trail length because hex mode fixes the per-section vertex count. Tuning H , $N_{\max}$, and quality thresholds gives predictable performance control.

XII. GLOBAL BOUNDEDNESS OF THE COUPLED SYSTEM

Theorem 7 (Global Boundedness Under Practical Assumptions). *Assume: (i) host impulses are bounded and v_k is clamped as implemented; (ii) corner dynamics use closed-form integration of a stable critically damped system; (iii) overshoot kicks are clamped and frame timestep satisfies practical bounds (i.e., no unbounded Δt); (iv) quality q is clamped to $[q_{\min}, 1]$ and history buffers have fixed capacity. Then all internal state variables of `velCursor` remain bounded for all time.*

TABLE I
SELECTED DEFAULTS FROM CURRENT IMPLEMENTATION.

Parameter	Value	Role
<code>trail.ttlMs</code>	420	trail lifetime (ms)
<code>trail.maxRects</code>	48	history capacity
<code>trail.ribbonSides</code>	6	strict hex resampling
<code>stackHex.concavityDepth</code>	0.16	concavity strength
<code>stackHex.partialEdgeShare</code>	0.82	nominal side sharing
<code>stackHex.turnPartialEdgeShare</code>	0.42	turn-cell side sharing
<code>overshoot.omega</code>	16	oscillator frequency (rad/s)
<code>overshoot.zeta</code>	0.16	damping ratio
<code>performance.qualityMin</code>	0.38	quality floor

Proof. Host velocity is bounded by Theorem 1. Corner offsets converge and remain bounded by Lyapunov stability. Overshoot is BIBO stable in continuous time and remains bounded in discrete time under practical timestep and bounded kick assumptions. Quality is clamped to a compact interval and is contractive toward a bounded target. History and scratch buffers are bounded by construction. Therefore the full coupled state remains bounded. $\square$

A. Plain interpretation

This is the engineering safety statement: there is no parameter path that can cause infinite values. Worst case, you degrade quality or clamp geometry, but you do not blow up the system.

XIII. IMPLEMENTATION DEFAULTS (CURRENT SNAPSHOT)

XIV. LIMITATIONS AND ENGINEERING RISKS

- The renderer depends on Monaco DOM contracts (e.g., `.cursor` element availability); upstream DOM changes can break sampling.
- Explicit Euler integration for the overshoot oscillator is stable in current parameter ranges but has known step-size sensitivity under extreme frame stalls or unbounded kicks.
- Geometric validity checks are local; global topological guarantees across long histories are not formally proven, though bounded history and local checks substantially reduce risk.
- Perceived smoothness remains hardware and display-rate dependent despite adaptive control.

A. Plain interpretation

The math makes the system robust, but the world can still break it: DOM changes, absurd frame stalls, or pathological geometry. The code mitigates these with clamping and fallback, but this is not a formal proof of correctness under arbitrary upstream changes.

XV. CONCLUSION

`velCursor` is a coupled interactive dynamics system: inertial command synthesis at the host layer and constrained polygonal rendering at the browser layer. The current implementation goes beyond a visual trail by enforcing geometric validity, turn-stability filtering, and adaptive quality control while maintaining near-caret attachment through a live head bridge and cap.

In addition to the code-faithful model, we established formal properties that matter for tuning and maintenance: bounded host velocity, Lyapunov-stable corner dynamics, BIBO-stable overshoot (with practical numerical conditions), contraction-stable quality regulation, and local validity guarantees for bounded concavity. Collectively, these results support systematic parameter adjustment rather than aesthetic trial-and-error.

ACKNOWLEDGMENT

This document is code-faithful to the current repository state and intended as a maintainable technical reference for future tuning and refactoring work.

REFERENCES

- [1] velCursor Repository, “`extension.js`,” command activation and configuration synchronization module.
- [2] velCursor Repository, “`cursorInertia.js`,” inertial cursor state machine and level mapping module.
- [3] velCursor Repository, “`cursorTrail.js`,” Monaco-side spring dynamics and hex-trail renderer.
- [4] velCursor Repository, “`package.json`,” command, keybinding, and configuration schema.